```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

# Programming Tutorial - 27

**4. Tutorial - Arrays, Encapsulation & Access**

Kuzey Aydınhan | 28. November 2025

## Contents


1. Overview

2. Arrays

3. Packages

4. Encapsulation

# Information

## Important

- 2nd Homework deadline 04. Dec @ 06:00
- Präsenzübung (Written Exam) on 21. Jan 2026
- Sign up for Übungsschein in Campus System until 10.12. 12:00

# Last Week

## Methods & Constructors

How to write methods, how to overload them

- Scanner usage
- Methods
- Constructors
- Method overloading
- Static methods

# Last Week

## Methods

```java
class Pool {
    int annualIncome;
    int annualExpense;

    int getNetIncome() {
        return annualIncome - annualExpense;
    }
}

```

```java
class Main {
    public static void main(String[] args) {
        Pool pool = new Pool();

        pool.annualIncome = 1302502;
        pool.annualExpense = 2938;

        int netIncome = pool.getNetIncome();
    }
}

```

# Last Week

## Static Methods

```java
class Pool {
    int annualIncome;
    int annualExpense;

    int getNetIncome() {
        return annualIncome - annualExpense;
    }

    static int calculateNetIncome(int income, int expense) {
        // Can't use this.income !!!
        return income - expense;
    }
}

```

```java
public static void main(String[] args) {
    // Pool pool = new Pool();
    // pool.annualIncome = 1302502;

    int income = 1302502;
    int expense = 2938;

    int netIncome = Pool.calculateNetIncome(income, expense);
}

```

**Important**

`Static` methods can't use non-static class attributes.

# Last Week

## Constructors

```java
class Pool {
    int annualIncome;
    int annualExpense;

    Pool() {
        // Default
    }

    Pool(int annualIncome, int annualExpense) {
        this.annualIncome = annualIncome;
        this.annualExpense = annualExpense;
    }

    // ...
}

```

```java
class Main {
    public static void main(Strin[] args) {
        // Pool pool = new Pool();
        // pool.annualIncome = 1390874;

        Pool pool = new Pool(1390874, 2398);

        int netIncome = pool.getNetIncome();
    }
}

```

# JavaDoc

## Documentation

- Documenting methods/classes
- Explain -in short- what it does
- Dont need to document private methods

- `@author`: Author of method/class (your u-kürzel)
- `@param`: What that method parameter is for
- `@return`: What the method returns
- `@throws`: If any exception can be thrown and why
- `@version`: Version of file (not used much)

**Programming Wiki**

You can find more information regarding the topic under: https://sdq.kastel.kit.edu/programmieren/JavaDoc

# JavaDoc

## Example

```
\**
 * Represents a car vehicle.
 *
 * @author uxxxx
 * @version 1.0.0.
 **\
class Car {
    String name;

    \**
     * Simulates car driving forward.
     *
     * @param person The person driving this car
     * @return true if car does not hit a wall
     * @throws IllegalArgumentException If the passed in person is null
     * @author uxxxx
     **\
    public boolean driveCar(Person person) {
        // ...
    }
}

```

# Homework

## Namings

**Bad Code:**

```java
String myLittlePony1 = "..";
String myLittlePony2 = "..";
String myLittlePony3 = "..";

boolean myLittlePony4 = myLittlePony1 + myLittlePony2..
// ...

int a = scanner.nextInt();
int b = scanner.nextInt();

```

Many have used **bad** names for variables / constants

- Don't use abbrevations when naming: `a, b, s, res..`
- Don't use different names: `myLittlePony..`

- Use readable names
- Use descriptive names

**Correct:**

```java
String firstInput = ..
String secondInput = ..
boolean isValid = ..

int result = ..

```

# Homework

## Methods

**Bad Code:**

```java
int first = scanner.nextInt();
int second = scanner.nextInt();

first = first.toLowerCase();
second = second.toLowerCase();

```

**Correct:**

```java
int first = scanner.nextInt().toLowerCase();
int second = scanner.nextInt().toLowerCase();

```

- You can add methods after eachother
- **Don't** redefine variables to add new method

# Homework

## If-Else

**Bad Code:**

```java
if (statement1) {
    System.out.println("Paper wins");
} else if (statement2) {
    System.out.println("Paper wins");
} else if (stement3) {
    System.out.println("Paper wins");
}

```

**Correct:**

```java
if (statement1 || statement2 || statement3) {
    System.out.println("Paper wins");
}

```

### Guidelines

You can check the guidelines for more: https://sdq.kastel.kit.edu/programmieren/

# Motivation

- Arrays & multidimension
- Packages
- Encapsulation principle
- Access modifiers

# Arrays

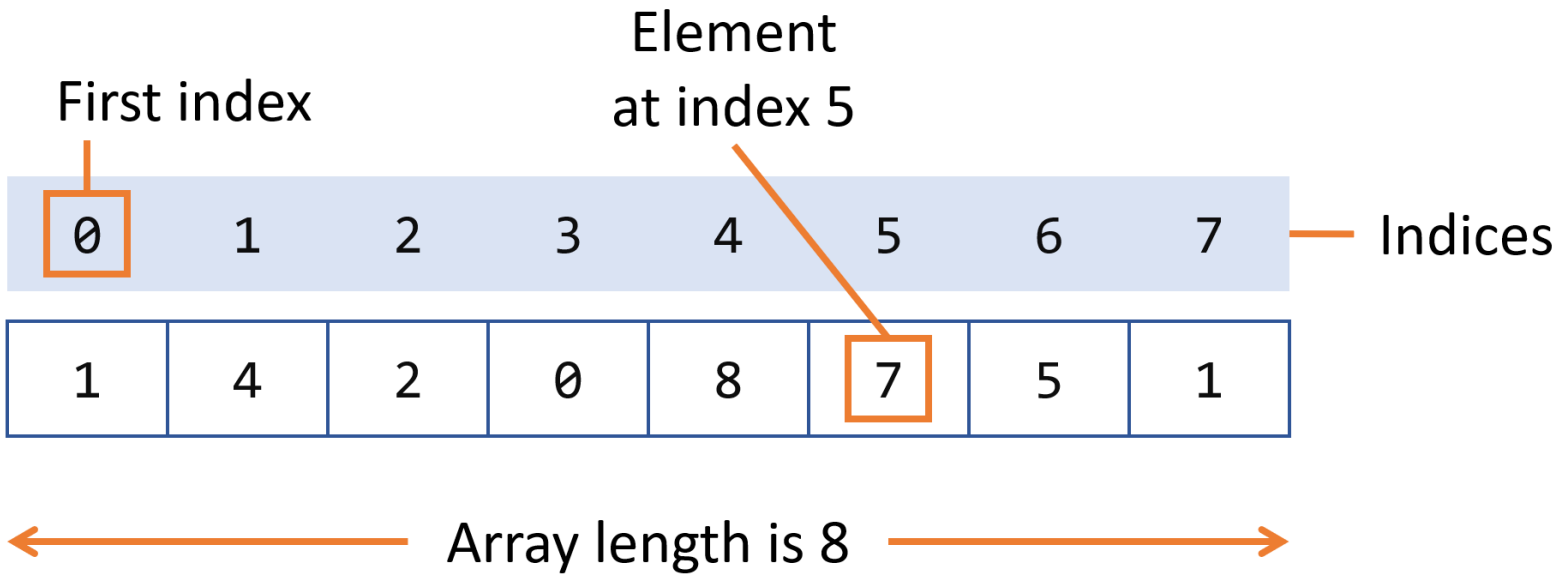
## Init

- Only one data type per array
- When initializing, each element will get the "default" value of type
- Get size of array: `array.length`
- 0-Indexed: { 0, 1, 2.. }

```java
int[] lateInit;
lateInit = new int[10];  // { 0, 0, 0, 0, 0.. }

int[] randomNumbers = new int[10];  // { 0, 0, 0, 0, 0.. }

int[] initialized = { 1, 5, 2, 3, 5 };

int length = randomNumbers.length;  // = 10

```

# Arrays

## Structure



# Arrays

- Get element at **i-th index**: `array[i]`
- Set element at **i-th index**: `array[i] = number`
- Last element in array: `length - 1`

```java
int[] initialized = { 1, 5, 2, 3, 4 };

int length = initialized.length;  // = 5
int lastElement = initialized[5 - 1];  // = 4
initialized[3] = 10;  // { 1, 5, 2, 10, 4 }

```

# Arrays

## Size

- Array lengths are **final** → can't resize
- Need more place? → New array
- Hence: Not suitable for **unknown lengths** → `Lists` later

```java
int[] array = new int[2];
array[0] = 0;
array[1] = 1;

array[2] = 2;  // ArrayIndexOutOfBoundsException

```

**Again!**

Arrays start at index 0!
Last element at `array.length - 1`

# Arrays

## Example

```java
int capacity = 10;
int currentIndex = 0;  // MAX 9 -> 0..9 = 10
Student[] students = new Student[capacity];  // { null, null, null, null.. }

boolean addStudent(Student student) {
    if (currentIndex < capacity) {

        students[currentIndex] = student;

        currentIndex++;

        return true;
    }
    return false;
}

```

- `{ null, null, null..null }`
- 10 elements
- `currentIndex = 0`

1. `addStudent(..)` :
   - `{ student1, null, null..  }`
   - `currentIndex = 1`
   - 10 elements
2. `addStudent(..)` :
   - `{ student1, student2, nul..}`
   - currentIndex = 2

# Arrays

## Iteration

- `for` loops
- `for-each` loops

```java
int[] array = { 1, 2, 3, 4, 5 };

for (int i=0; i<array.length; i++) {
    array[i];
}

for (int currentNumber : array) {
    currentNumber;
}

```

# Exercise 1

Method 1:

- Write a method that will take in an integer-array
- If array empty or `null` print 0
- And print out the integers (with their signs) as a single line
- Example: { 1, 24, 3, -5 } → "1, 24, 3, -5"
- Useful: `System.out::print`

Method 2:

- Write a method that will take in an integer-array
- And print out the middle value between max and min of the array
- If array empty or `null` print 0
- Example: { 10, -12, 14 } → max=14, min=-12, result=1

# Multidimensional Arrays

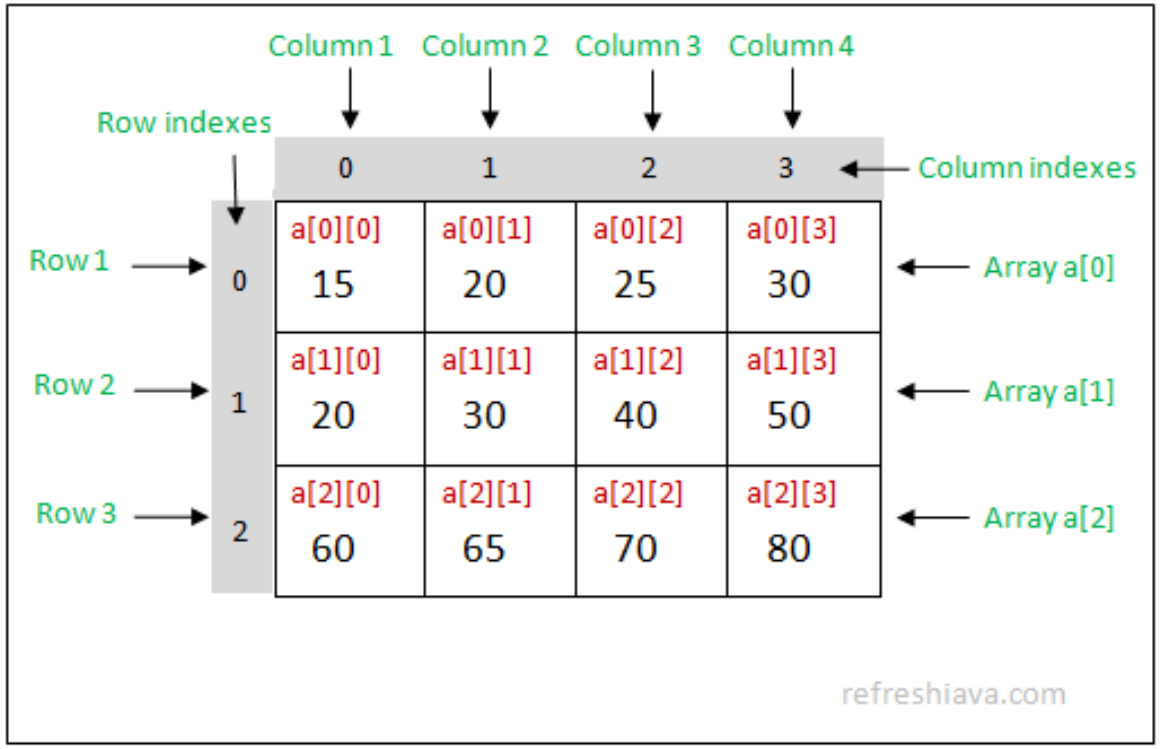
```java
Tile[][] gameBoard = new Tile[2][3];
fillTiles();

// {
// { T, T, T },
// { T, T, T }
// }

```

- Useful for "game boards" (2D)
- For rest not that much
- `int[][] array`
  `= new int [rowSize] [columnSize]`

## Problems

- Indexing gets hard
- What does `int[][][][]` mean?

# Multidimensional Arrays



# Multidimensional Arrays

## Iteration

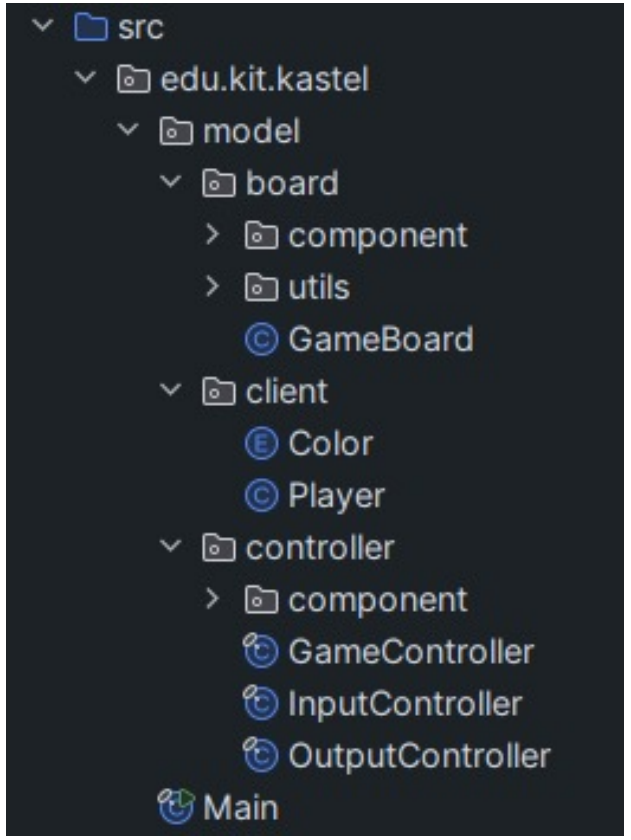
```java
Tile[][] gameBoard = new Tile[2][3];

for (int rowIndex=0; rowIndex<gameBoard.length; rowIndex++) {

    Tile[] currentRow = gameBoard[rowIndex];

    for (int columnIndex=0; columnIndex<currentRow.length; columnIndex++) {
        Tile currentTile = currentRow[columnIndex];

        // = gameBoard[rowIndex][columnIndex];
    }
}

for (Tile[] row : gameBoard) {
    for (Tile currentTile : row) {
        // ...
    }
}

```

### Exercise

You can revisit this exercise again: https://github/tut2/exercise3

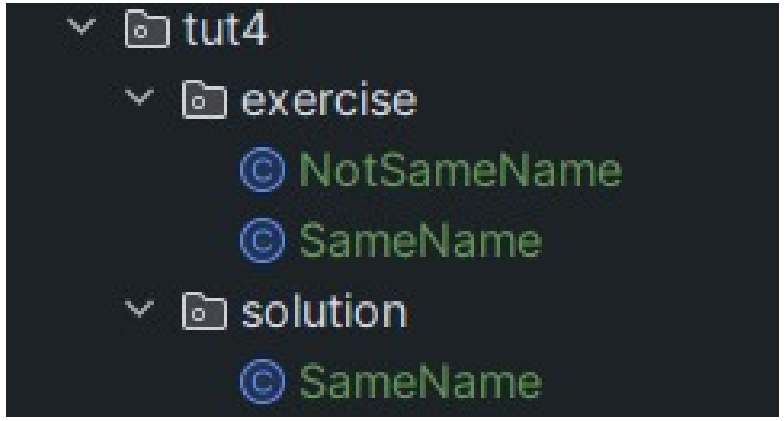
# Packages

- Better structure
- Readible code
- Grouping similar classes



# Packages

- Should not contain same names
- Each name in a package is unique to class
- However the same name can be used in **another** package
- Package names should be lower case



# Packages

## Import

Two ways of import

- Full usage: `java.awt.Date date`
- Class import: `import java.awt.Date;`

```java
import java.utils.Scanner;

import java.math.*;  // Bad practice

void main() {
    Scanner scanner = new Scanner();

    java.aws.Date date = new java.aws.Date();
}

```

**Default**

`java.lang` package is always implicitly imported → no need to use import statement

# Encapsulation

## Idea

- Attributes are hidden from outside
- Can not be modified directly
- Use `getters / setters` instead

**Bad Code:**

```
Car car = new Car("mercedes", "green", "c200");

car.name = "bmw";
car.model = "i200";
String color = car.color;

```

**Correct:**

```
Car car = new Car("mercedes", "green", "c200");

car.setName("bmw");
car.setModel("i200");
String color = car.getColor();

```

# Encapsulation

## Access Modifiers

Using access modifiers, we can limit or extend the access.

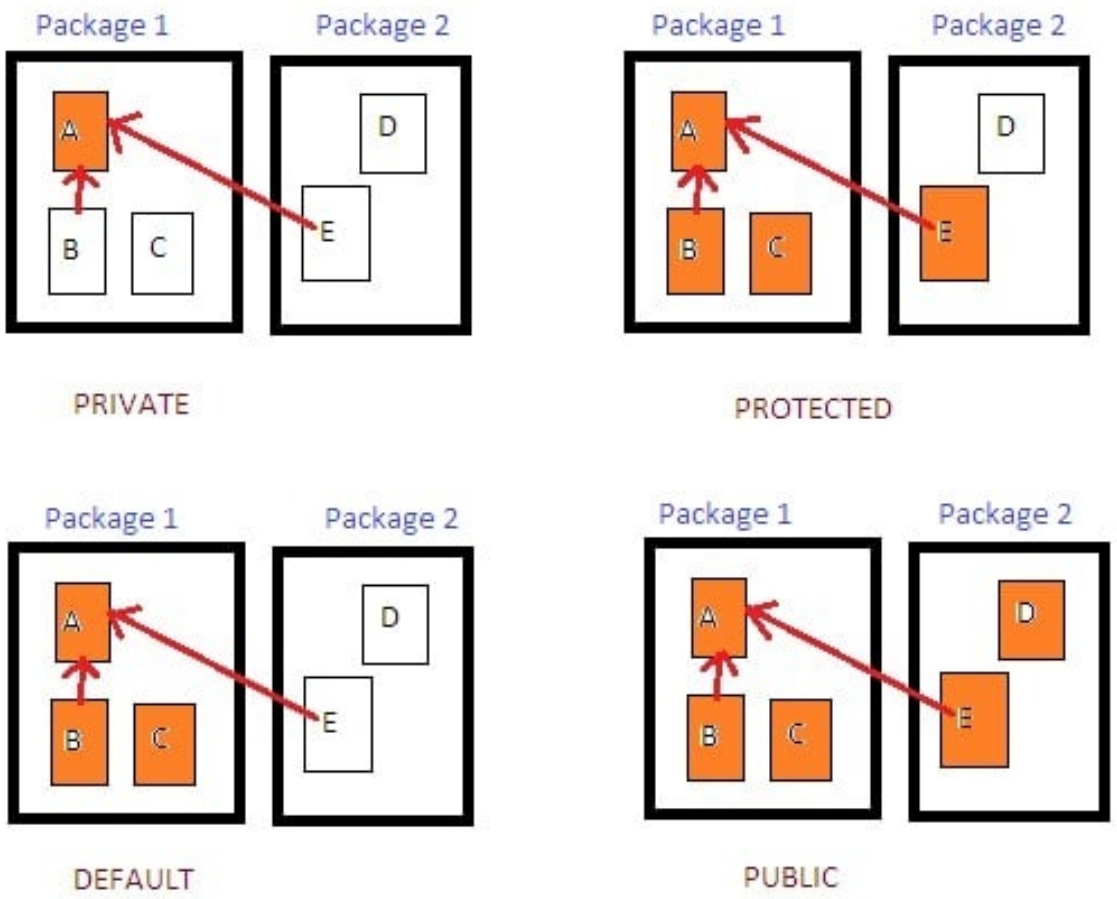
- **`public:`** Element is for all classes visible
- **`private:`** Element is only inside its own class visible
- **`protected:`** Element is also for its subclasses and other classes in same package visible
- **`package-private:`** Element is only from the classes in same package visible

# Encapsulation

## Access Modifiers

| Modifier | Class | Package | SubClass | Global |
|---|---|---|---|---|
| Public | ✓ | ✓ | ✓ | ✓ |
| Protected | ✓ | ✓ | ✓ | ✗ |
| Default | ✓ | ✓ | ✗ | ✗ |
| Private | ✓ | ✗ | ✗ | ✗ |

# Encapsulation



# Convention

From now on:

- Classes are generally **`public`**
- Constructors are generally **`public`**
- Attributes of classes are always **`private`**
- Constants can be -only if necessary- **`public`**, usually **`private`**
- Getter/Setter methods are always **`public`**

**Important**

JavaDoc has to be written on **`public`** methods and **all** classes.

# How Does It Look?

```java
public class Circle {
    private static final PI = 3.14;  // Constant

    private float radius;
    private float diameter;
    private float circumference;

    public Circle(float radius) {
        this.radius = radius;
    }

    public void setRadius(float radius) {
        this.radius = radius;
    }

    public float getRadius() {
        return this.radius;
    }

    // ...
}

```

- Make every attribute **`private`**
- Add **getters & setters** for each attribute

# Encapsulation

## Avoid

**Bad Code:**

```java
private class Student {
    public String name;
    public String id;

    // ...
}

```

**Correct:**

```java
// package-private
class Student {
    private String name;
    private String id;

    // Getters / Setters
}

```

- Never use **`private`** for classes
- If you want to make it **`package-private`**: no modifier
- Never make the attributes **`public`**

# Other Modifiers

- **`final:`** The variable can't be modified once set
- **`static:`** The variable doesn't belong to class attributes
- **`static final:`** Constants
- Constants are to be written in `UPPER_CASE`

```java
public class Pool {
    private static final int CAPACITY = 25;
    private final String name;
    private Student[] students = new Student[CAPACITY];

    public Pool(String name) {
        this.name = name;
    }

    public doSomething() {
        this.name = "something";  // Not allowed
    }

    public static int getCapacity() {
        return CAPACITY;
    }
}

```

# Exercise 2

## What is wrong?

```java
private class Cone {
    static final double pi = Math.PI;
    Circle base;
    double height;
    double slantHeight;

    public Cone(Circle base, Double height) {
        this.base = base;
        this.height = height;
        this.slantHeight = getSlantHeight(base, height);
    }

    private double getSlantHeight(Circle base, double height) {
        return Math.hypot(base.getRadius(), height);
    }

    private double getSlantHeight() {
        return this.slantHeight;
    }

    private double getHeight() {
        return this.height;
    }
}

```

# Conclusion

- Arrays
- Multidimensional arrays
- Encapsulation
- Access modifiers

# Solution

## Exercise 1

```java
void printIntegers(int[] array) {
    if (array == null || array.length == 0) {
        System.out.println(0);
        return;
    }

    for (int current : array) {
        System.out.print(current + ", ");
    }
    System.out.println();
}

```

```java
void printMid(int[] array) {
    if (array == null || array.length == 0) {
        System.out.println(0);
        return;
    }

    int max = array[0];
    int min = array[0];

    for (int current : array) {
        if (current > max) {
            max = current;
        } else if (current < min) {
            min = current;
        }
    }
    System.out.println((min + max) / 2);
}

```

# Solution

## Exercise 2

```java
private class Cone {
    static final double pi = Math.PI;
    Circle base;
    double height;
    double slantHeight;

    public Cone(Circle base, Double height) {
        this.base = base;
        this.height = height;
        this.slantHeight = getSlantHeight(base, height);
    }

    private double getSlantHeight(Circle base, double height) {
        return Math.hypot(base.getRadius(), height);
    }

    private double getSlantHeight() {
        return this.slantHeight;
    }

    private double getHeight() {
        return this.height;
    }
}
```

# Solution

## Exercise 2

The problems:

- Class definition must not be `private`
- Constants should be written in `UPPER_CASE`
- Attributes and the constant must be `private`
- Constructor parameter `height` should be primitive or include `null`-check
- `getSlantHeight(Circle base, double height)` can be static or use class attributes
- `getSlantHeight()` must not be private
- `getHeight()` must not be private
- `base, height` and `slantHeight` remain unchanged → should be `final`

# Solution

## Exercise 2

```java
class Cone {
    private static final double PI = Math.PI;
    private final Circle base;
    private final double height;
    private final double slantHeight;

    public Cone(Circle base, double height) {
        this.base = base;
        this.height = height;
        this.slantHeight = getSlantHeight(base, height);
    }

    private static double getSlantHeight(Circle base, double height) {
        return Math.hypot(base.getRadius(), height);
    }

    public double getHeight() {
        return height;
    }

    public double getSlantHeight() {
        return slantHeight;
    }
}

```